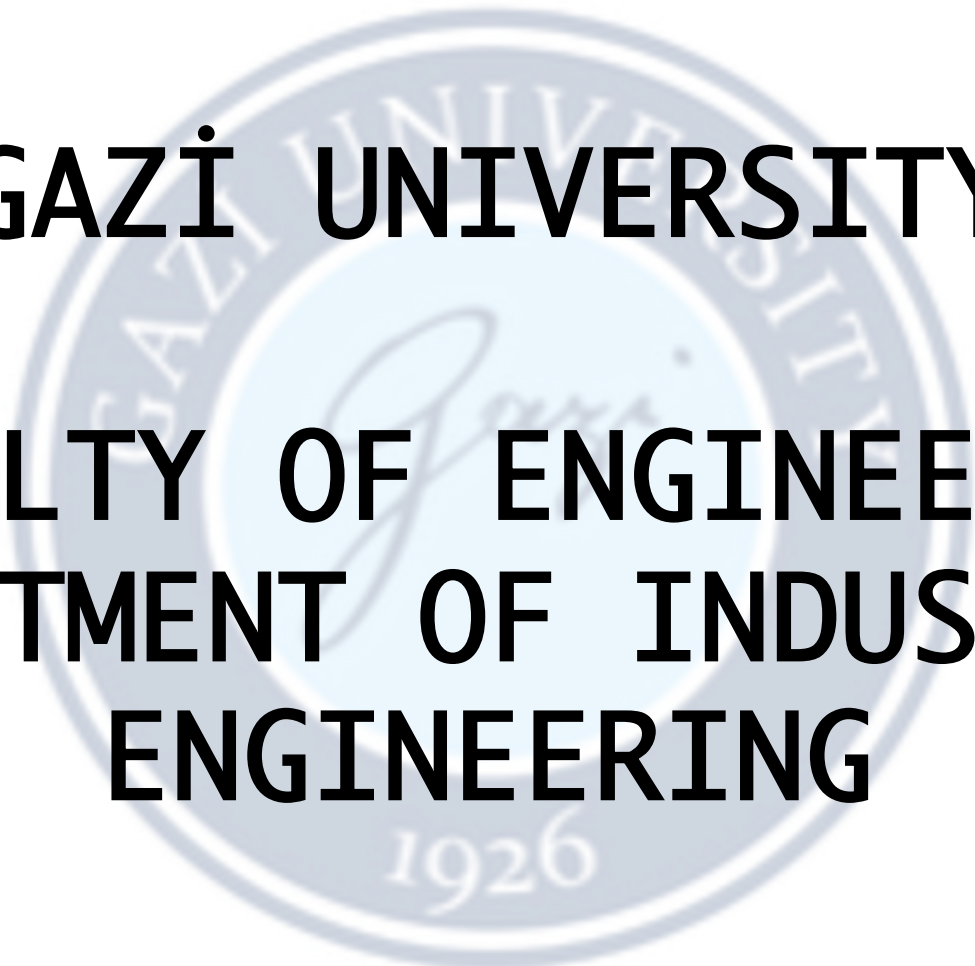


The seal of Gazi University is a circular emblem. It features a blue outer ring with the text "GAZİ ÜNİVERSİTESİ" in white. Inside the ring is a white circle containing a blue silhouette of a person, likely a historical figure, and the year "1926" at the bottom.

GAZİ UNIVERSITY

**FACULTY OF ENGINEERING
DEPARTMENT OF INDUSTRIAL
ENGINEERING**

Lecturer : Dr Ercan Ezin

IE326-DATABASE MANAGEMENT SYSTEMS

WEEK 13: GROUPING DATA-MODIFY DATA

GROUPING DATA



GROUP BY, HAVING

GROUP BY

The `GROUP BY` clause divides the rows returned from the `SELECT` statement into groups.

For each group, you can apply an aggregate function such as `SUM()` to calculate the sum of items or `COUNT()` to get the number of items in the groups.

The following illustrates the basic syntax of the `GROUP BY` clause:

```
SELECT
    column_1,
    column_2,
    ...,
    aggregate_function(column_3)
FROM
    table_name
GROUP BY
    column_1,
    column_2,
    ...;
```

In this syntax:

- First, select the columns that you want to group such as `column1` and `column2`, and column that you want to apply an aggregate function (`column3`).
- Second, list the columns that you want to group in the `GROUP BY` clause.

GROUP BY

The `GROUP BY` clause divides the rows by the values in the columns specified in the `GROUP BY` clause and calculates a value for each group.

It's possible to use other clauses of the `SELECT` statement with the `GROUP BY` clause.

PostgreSQL evaluates the `GROUP BY` clause after the `FROM` and `WHERE` clauses and before the `HAVING`, `SELECT`, `DISTINCT`, `ORDER BY` and `LIMIT` clauses.



GROUP BY-EXAMPLE-DISTINCT

payment
* payment_id
customer_id
staff_id
rental_id
amount
payment_date

The following example uses the GROUP BY clause to retrieve the customer_id from the payment table:

```
SELECT customer_id FROM payment GROUP BY customer_id ORDER BY customer_id;
```

Each customer has one or more payments. The GROUP BY clause removes duplicate values in the customer_id column and returns distinct customer ids.

In this example, the GROUP BY clause works like the DISTINCT operator.



customer_id

1
2
3
4
5
6
7
8
...

GROUP BY-EXAMPLE-SUM()

The **GROUP BY** clause is useful when used in conjunction with an aggregate function. The following query uses the **GROUP BY** clause to retrieve the total payment paid by each customer:

```
SELECT customer_id, SUM (amount)
FROM payment
GROUP BY customer_id
ORDER BY customer_id;
```



payment
* payment_id
customer_id
staff_id
rental_id
amount
payment_date

customer_id	sum
1	114.70
2	123.74
3	130.76
4	81.78
5	134.65
6	84.75
7	130.72
...	

In this example, the **GROUP BY** clause groups the payments by the customer id. For each group, it calculates the total payment.

GROUP BY-EXAMPLE-SUM()

The following statement uses the ORDER BY clause with GROUP BY clause to sort the groups by total payments:

```
SELECT customer_id, SUM (amount) as TOTAL  
FROM payment  
GROUP BY customer_id  
ORDER BY TOTAL DESC;
```



payment
* payment_id
customer_id
staff_id
rental_id
amount
payment_date

customer_id	sum
148	211.55
526	208.58
178	194.61
137	191.62
144	189.60

HAVING

The `HAVING` clause specifies a search condition for a group. The `HAVING` clause is often used with the `GROUP BY` clause to filter groups based on a specified condition.

The following statement illustrates the basic syntax of the `HAVING` clause:

```
SELECT
    column1,
    aggregate_function (column2)
FROM
    table_name
GROUP BY
    column1
HAVING
    condition;
```

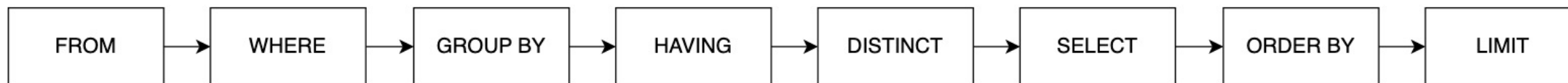
In this syntax:

- First, the `GROUP BY` clause groups rows into groups by the values in the `column1`.
- Then, the `HAVING` clause filters the groups based on the `condition`.

HAVING

Besides the `GROUP BY` clause, you can also include other clauses such as `JOIN` and `LIMIT` in the statement that uses the `HAVING` clause.

PostgreSQL evaluates the `HAVING` clause after the `FROM`, `WHERE`, `GROUP BY`, and before the `DISTINCT`, `SELECT`, `ORDER BY` and `LIMIT` clauses:



Because PostgreSQL evaluates the `HAVING` clause before the `SELECT` clause, you cannot use the column aliases in the `HAVING` clause.

This restriction arises from the fact that, at the point of `HAVING` clause evaluation, the column aliases specified in the `SELECT` clause are not yet available.

HAVING VS WHERE

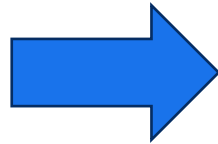
The `WHERE` clause filters the rows based on a specified condition whereas the `HAVING` clause filter groups of rows according to a specified condition.

In other words, you apply the condition in the `WHERE` clause to the rows while you apply the condition in the `HAVING` clause to the groups of rows.

HAVING EXAMPLE-SUM()

The following query uses the `GROUP BY` clause with the `SUM()` function to find the total payment of each customer:

```
SELECT
  customer_id,
  SUM (amount) amount
FROM
  payment
GROUP BY
  customer_id
ORDER BY
  amount DESC;
```



customer_id	amount
148	211.55
526	208.58
178	194.61
137	191.62
...	

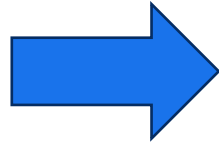
payment

* payment_id
customer_id
staff_id
rental_id
amount
payment_date

HAVING EXAMPLE-SUM()

The following statement adds the `HAVING` clause to select the only customers who have been spending more than `200`:

```
SELECT
  customer_id,
  SUM (amount) amount
FROM
  payment
GROUP BY
  customer_id
HAVING
  SUM (amount) > 200
ORDER BY
  amount DESC;
```



customer_id	amount
148	211.55
526	208.58

(2 rows)

payment

* payment_id
customer_id
staff_id
rental_id
amount
payment_date

HAVING EXAMPLE-COUNT

The following query uses the GROUP BY clause to find the number of customers per store:

```
SELECT
  store_id,
  COUNT (customer_id)
FROM
  customer
GROUP BY
  store_id;
```



store_id	count
1	326
2	273

(2 rows)

The statement on the right adds the HAVING clause to select a store that has more than 300 customers:

```
SELECT
  store_id,
  COUNT (customer_id)
FROM
  customer
GROUP BY
  store_id
HAVING
  COUNT (customer_id) > 300;
```



store_id	count
1	326

(1 row)

customer	
* customer_id	
store_id	
first_name	
last_name	
email	
address_id	
activebool	
create_date	
last_update	
active	

EXERCISES- PREPARE YOUR LOCAL TABLE

```
-- Cleanup existing schema
DROP TABLE IF EXISTS order_details;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;

-- 1. Create Tables
CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50),
    unit_price NUMERIC(10, 2)
);

CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE DEFAULT CURRENT_DATE,
    status VARCHAR(20)
);

CREATE TABLE order_details (
    detail_id SERIAL PRIMARY KEY,
    order_id INT REFERENCES orders(order_id),
    product_id INT REFERENCES products(product_id),
    quantity INT CHECK (quantity > 0)
);
```

```
-- 2. Insert Seed Data
INSERT INTO products (product_name, category, unit_price) VALUES
('Steel Sheet', 'Raw Material', 100.00),
('Copper Wire', 'Raw Material', 50.00),
('Gear Assembly', 'Component', 250.00),
('Motor', 'Component', 500.00),
('Welding Rod', 'Consumable', 15.00);

INSERT INTO orders (customer_id, order_date, status) VALUES
(501, '2024-01-10', 'Shipped'),
(502, '2024-01-12', 'Pending'),
(501, '2024-01-15', 'Shipped'),
(503, '2024-02-01', 'Shipped');

INSERT INTO order_details (order_id, product_id, quantity) VALUES
(1, 1, 10), (1, 3, 2), -- Order 1
(2, 4, 1),           -- Order 2
(3, 2, 20), (3, 3, 5), -- Order 3
(4, 5, 100);         -- Order 4
```

MODIFYING DATA



INSERT, UPDATE, DELETE, TRANSACTIONS

INSERT

The `INSERT` statement is used to add new rows of data to a table. You can insert a single row, multiple rows, or even data from another table.

1. Basic Syntax (Single Row)

To insert a row, specify the table name and the columns you want to fill, followed by the values.

SQL



```
INSERT INTO products (product_name, category, unit_price)
VALUES ('Induction Motor', 'Component', 750.00);
```

Tip: If inserting values for **all** columns in the exact order they were created, the column names can be omitted. However, explicitly naming columns is safer for long-term code maintenance.

INSERT

INSERTING MULTIPLE ROWS

PostgreSQL allows inserting multiple rows in a single command, which is more efficient than running multiple separate queries.

SQL



```
INSERT INTO products (product_name, category, unit_price)
VALUES
    ('Valve Spring', 'Component', 12.50),
    ('Hydraulic Oil', 'Consumable', 45.00),
    ('Aluminum Plate', 'Raw Material', 85.00);
```

If a column has a `DEFAULT` value (like a `SERIAL` ID or a timestamp) or allows `NULL`, it can be omitted from the list.

INSERTING DEFAULT VALUES

SQL



```
-- order_date will automatically use the default value (e.g., CURRENT_DATE)
INSERT INTO orders (customer_id, status)
VALUES (505, 'Processing');
```

INSERT

The `RETURNING` clause is a PostgreSQL-specific feature that returns the data generated by the insert (such as an auto-incremented ID) without requiring a separate `SELECT` query.

RETURNING CLAUSE

SQL



```
INSERT INTO products (product_name, category, unit_price)
VALUES ('Industrial Sensor', 'Electronics', 120.00)
RETURNING product_id;
```

UPDATE

The `UPDATE` statement is used to modify existing records in a table. It is typically used with a `WHERE` clause to target specific rows.

To update a specific record, identify the table, the column to change, and the condition to locate the row.

SQL



```
UPDATE products
SET unit_price = 110.00
WHERE product_id = 1;
```

Tip: Always verify the `WHERE` clause before executing. If it is omitted, **all rows** in the table will be updated.

UPDATE

The `UPDATE` statement is used to modify existing records in a table. It is typically used with a `WHERE` clause to target specific rows.

To update a specific record, identify the table, the column to change, and the condition to locate the row.

SQL



```
UPDATE products  
SET unit_price = 110.00  
WHERE product_id = 1;
```

Tip: Always verify the `WHERE` clause before executing. If it is omitted, **all rows** in the table will be updated.

UPDATE

- ★ Multiple columns can be updated at once by separating the column-value pairs with commas.

SQL



```
UPDATE products
SET category = 'Refined Material',
    unit_price = 55.00
WHERE product_id = 2;
```

- ★ Updates can be performed using calculations based on existing values, such as applying a price increase.

SQL



```
-- Increase all prices in the 'Component' category by 10%
UPDATE products
SET unit_price = unit_price * 1.10
WHERE category = 'Component';
```

UPDATE

Similar to the `INSERT` statement, PostgreSQL allows the use of `RETURNING` to see the modified rows immediately.

SQL



```
UPDATE orders
SET status = 'Completed'
WHERE order_id = 102
RETURNING order_id, status;
```

UPDATE-Exercises

Q1. Basic Update: Write a query to change the status of order `101` to 'Delivered'.

SQL



```
UPDATE orders
SET status = 'Delivered'
WHERE order_id = 101;
```

Q2. Conditional Mass Update: Adjust the `unit_price` of all 'Raw Material' products by subtracting 5.00 from their current price.

SQL



```
UPDATE products
SET unit_price = unit_price - 5.00
WHERE category = 'Raw Material';
```

UPDATE-Exercises

Q3. Multi-column with Verification: Change the `product_name` to 'Heavy Motor' and `unit_price` to 600.00 for the product with `product_id = 4` . Return the updated row data.

SQL



```
UPDATE products
SET product_name = 'Heavy Motor',
    unit_price = 600.00
WHERE product_id = 4
RETURNING *;
```

DELETE

The `DELETE` statement is used to remove one or more rows from a table. Like `UPDATE`, it typically requires a `WHERE` clause to avoid unintended data loss.

To remove specific records, specify the table and the condition that identifies the rows to be deleted.

SQL



```
DELETE FROM products  
WHERE product_id = 5;
```

Tip: If the `WHERE` clause is omitted, **all rows** in the table will be deleted, though the table structure itself will remain.

DELETE

- ★ PostgreSQL allows you to return the data of the rows that were deleted. This is useful for archiving or verifying the operation.

SQL



```
DELETE FROM products
WHERE category = 'Consumable'
RETURNING *;
```

- ★ PostgreSQL does not use a direct `DELETE JOIN` syntax like some other SQL dialects. Instead, it uses the `USING` clause to join other tables for filtering criteria.

SQL



```
-- Delete all order details for products categorized as 'Raw Material'
DELETE FROM order_details
USING products
WHERE order_details.product_id = products.product_id
AND products.category = 'Raw Material';
```

DELETE

`CASCADE` is a property of a **Foreign Key constraint**, not a standalone command. When a table is created with `ON DELETE CASCADE`, deleting a row in the parent table automatically removes all related rows in the child table.

Example Setup:

SQL



```
CREATE TABLE order_details (  
    detail_id SERIAL PRIMARY KEY,  
    order_id INT REFERENCES orders(order_id) ON DELETE CASCADE,  
    product_id INT REFERENCES products(product_id)  
);
```

Tip: With `ON DELETE CASCADE` enabled, running `DELETE FROM orders WHERE order_id = 101;` will automatically delete all rows in `order_details` that belong to order 101.

DELETE

Exercises

Q1. Basic Delete: Write a query to delete the product with `product_id = 3`.

SQL



Q2. Filtered Delete: Remove all orders that have a status of 'Cancelled'.

SQL



Q3. Delete with Using Delete all rows from the `order_details` table that are associated with `customer_id = 501`. (Note: `customer_id` is in the `orders` table).

SQL



DELETE-DUPLICATE ROWS

In real-world databases, duplicate data can occur due to missing constraints or improper data imports. PostgreSQL provides several ways to identify and remove these duplicates while keeping one unique instance.

To find duplicates, use `GROUP BY` and `HAVING` on the columns that should be unique.

SQL



```
SELECT product_name, category, COUNT(*)  
FROM products  
GROUP BY product_name, category  
HAVING COUNT(*) > 1;
```

DELETE-DUPLICATE ROWS-CTID

Every row in PostgreSQL has an internal, hidden column called `ctid` that represents the physical location of the row version. It is the simplest way to distinguish between two rows that have identical data.

SQL



```
DELETE FROM products a
USING products b
WHERE a.ctid < b.ctid
      AND a.product_name = b.product_name
      AND a.category = b.category;
```

Tip: In this query, we compare rows `a` and `b`. If they have the same data but different `ctid` values, the one with the smaller `ctid` is deleted, leaving the "newest" version.

TRANSACTIONS

A transaction is a single unit of work that contains one or more SQL statements. Transactions follow the **ACID** principle (Atomicity, Consistency, Isolation, Durability), ensuring that either all operations succeed or none of them are applied.

Basic Commands

- **BEGIN** : Starts a new transaction.
- **COMMIT** : Saves all changes made during the transaction to the database.
- **ROLLBACK** : Undoes all changes made since the **BEGIN** command.

TRANSACTIONS-STANDART WORKFLOW

A transaction is typically used when multiple related tables must stay in sync. For example, when a product is ordered, you must create an order record and decrease the product stock simultaneously.

SQL



```
BEGIN;

-- Step 1: Create the order
INSERT INTO orders (customer_id, status)
VALUES (700, 'Shipped');

-- Step 2: Record the details
INSERT INTO order_details (order_id, product_id, quantity)
VALUES (105, 1, 5);

-- Step 3: Finalize the changes
COMMIT;
```

TRANSACTIONS-ROLLBACK

If an error occurs during any step of the transaction (e.g., a product is out of stock), you can cancel the entire process to prevent partial data entry.

SQL



```
BEGIN;
```

```
UPDATE products
```

```
SET unit_price = unit_price + 10
```

```
WHERE category = 'Component';
```

```
-- If you realize you made a mistake (e.g., wrong category)
```

```
ROLLBACK;
```

Tip: In many database tools and drivers, if a single statement within a `BEGIN/COMMIT` block fails, the entire transaction is automatically marked as aborted and must be rolled back.

TRANSACTIONS-SAVEPOINTS

Savepoints allow you to roll back part of a transaction instead of the whole thing.

SQL



```
BEGIN;

INSERT INTO products (product_name, category) VALUES ('New Tool', 'Equipment');
SAVEPOINT sp1;

INSERT INTO products (product_name, category) VALUES ('Experimental Tool', 'Test');
-- Something went wrong with the experimental tool
ROLLBACK TO SAVEPOINT sp1;

-- This will only commit 'New Tool'
COMMIT;
```

THE END



CHECK OUT COURSE WEBSITE: [HTTPS://GAZI-END-MUH.GITHUB.IO](https://GAZI-END-MUH.GITHUB.IO)

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM